Pair programming	Complex Panini rule implementation with 2 engineers + 1 linguist	40% reduction in rule errors
Test-driven development (TDD)	Write test first for each morphological rule before implementation	Higher confidence in accuracy
Small releases	Weekly release of API improvements to staging	Faster feedback from linguists
Continuous integration	GitHub Actions runs all tests on every commit	Early defect detection (avg 2 hours)
Refactoring	Monthly code cleanup session	Code duplication reduced from 7% to 4%
Collective code ownership	Any engineer can fix any bug; no silos	Faster bug resolution (avg 6 hours)
On-site customer	Sanskrit linguist available full-time during sprints	Immediate clarification of rules
Sustainable pace	40-hour weeks; no overtime except critical fixes	Consistent velocity (22-28 points)

4.4.2 Small Release Strategy

Table 4.16: Small Release Plan

Release	**Content**	**Frequency**
v0.1.0 (Alpha)	Basic morphological analyzer (100 sutras)	Week 4
v0.2.0 (Alpha)	Sandhi splitting for common patterns	Week 6
v0.3.0 (Beta)	Semantic parser for simple verses	Week 8
v0.4.0 (Beta)	Knowledge extraction module	Week 10
v0.5.0 (Beta)	REST API with 3 endpoints	Week 12
v1.0.0 (Production)	Complete framework with documentation	Week 18

4.4.3 Scrum Framework Integration

Table 4.17: Scrum + XP Integration

Scrum Element	XP Practice Integrated	Outcome
Sprint planning	User stories with acceptance tests (TDD)	Clear definition of done
Daily scrum	Pair programming coordination	Better task allocation
Sprint review	Small release demo to stakeholders	Immediate feedback
Sprint retrospective	Refactoring and process improvement planning	Continuous improvement
Product backlog	Continuous prioritization by linguist (on-site customer)	Highest value delivered first

4.4.4 Lean Principles Applied

Table 4.18: Lean Principles in Sanskrit NLP Project

Lean Principle	Application	Measured Impact
Eliminate waste	Auto-generate documentation from code; no duplicate rule specification	Saved 15 person-hours per sprint
Amplify learning	Weekly knowledge transfer sessions between linguists and engineers	Reduced rule misinterpretation by 60%
Decide as late as possible	API design finalized after 3 sprints of experimentation	Avoided 2 major redesigns
Deliver as fast as possible	CI/CD pipeline deploys to staging within 1 hour of commit	Lead time reduced from 5 days to 4 hours
Empower the team	Linguists can directly fix rule mapping via config files	No dependency on engineers for rule changes
Build integrity in	Automated sandhi validation before merge	Defect escape rate reduced to 12%
See the whole	Burndown chart + CFD visible to all stakeholders	Improved transparency and trust

4.4.5 Scrum, XP, and Lean Comparison